Teaching an AI to Answer Questions in Khmer

A fine-tuning report on Gemma 3 and Gemma 4 with LoRA · Author: Hang Muykhorng · November 2026
Notebook: colab.research.google.com/drive/13HhuzTUBf2z0j1YvtKlAnOdQCDDcM_6l

Summary

We took Google's open AI model **Gemma**, which can already hold a conversation, and gave it extra training on Khmer question-and-answer examples. This is called **fine-tuning**. We ran two experiments: one teaching the AI facts from a bank's FAQ page, and one teaching it to explain general topics in a short, simple way.

36%	~8 min	< 3%	0
lower error on unseen bank questions after training	training time per experiment on one GPU	of the model's internal numbers had to be trained	experiments that showed overfitting (memorising)

KEY TAKEAWAYS

- **It works.** Before training, the AI invented a wrong answer about a bank card (it said the card was a medicine). After training, it gave the correct answer from the bank's FAQ.
- **It is cheap.** Each experiment finished in about 8 minutes, using a fraction of one GPU's memory, because only a small add-on part of the model is trained.
- **It learned instead of memorising.** Its error on questions it had never seen kept going down throughout training.
- **There is room to improve.** The second experiment was still improving when training stopped. Training longer, on more data, should make it better.

How to read this report

If you are...	Read
A manager or non-technical reader	This summary, section 1 (the problem), section 7 (before vs after answers) and section 9 (what it means)
A student or teacher	Everything. Section 2 explains every technical term in plain words.
An engineer	Sections 4–6 and the appendix (exact settings and loss values). The grey *Technical detail* boxes give the code-level facts.

Throughout the report, green **In plain words** boxes give the simple explanation, and grey **Technical detail** boxes give the precise version.

Contents

1. The problem we are solving
2. Key ideas explained
3. How the process works
4. The data
5. The training settings
6. Results: did the AI learn?
7. Before vs after: real answers
8. Cost and resources
9. What this means
10. Limitations and issues found
11. Recommended next steps
Appendix: full numbers

1. The problem we are solving

Modern AI chat models are trained on huge amounts of internet text, much of it in English. As a result, they often:

- **Don't know local or company-specific facts**, such as the products and fees of a Cambodian bank.
- **Make up confident-sounding answers** when they don't know something. This is called *hallucination*.
- **Write Khmer less reliably** than English, sometimes mixing in other languages.

Building a new AI from scratch would cost millions of dollars. The practical alternative is **fine-tuning**: take an existing model and give it a short course of extra training on our own examples.

> **IN PLAIN WORDS**
> Think of a new employee who is already well educated. We don't send them back to school. We give them the company handbook to study for a day. Fine-tuning is the AI version of that.

The two experiments

	Experiment 1: Bank FAQ	Experiment 2: General Khmer Q&A
Question asked	Can the AI learn new facts it doesn't know?	Can the AI learn a new way of answering?
Teaching material	514 questions and answers from ACLEDA Bank's FAQ	5,000 general questions with short, simple answers
AI model	Gemma 3, 4 billion parameters (smaller)	Gemma 4, 12 billion parameters (larger)

2. Key ideas explained

This section explains every technical term used in the report. Each one has a plain-language meaning and a short technical note.

Term	In plain words	Technical note
Large language model (LLM)	An AI that has read a huge amount of text and learned to predict which word comes next. That is how it writes answers.	A transformer neural network that predicts the next token.
Gemma 3 / Gemma 4	Free AI models from Google that anyone can download and customise.	Open-weight models. We used the instruction-tuned (`-it`) versions, which already follow chat instructions.
Parameters / weights	The billions of internal numbers that store what the model knows. "4B" means 4 billion of them.	The learned values of the network's matrices.
Fine-tuning	Giving an existing AI extra training on our own examples so it gets better at one job.	Supervised fine-tuning (SFT) on question → answer pairs.
LoRA and PEFT	Instead of changing all the AI's numbers, we attach a small add-on and train only that. It's faster, cheaper and needs less memory.	Low-Rank Adaptation, a kind of Parameter-Efficient Fine-Tuning. See figure 1.
4-bit quantization	Storing the model's numbers in a compressed form, like a smaller photo file, so it fits on a cheaper GPU.	Weights stored in 4-bit precision (QLoRA-style) using `bitsandbytes`.
GPU	A powerful graphics chip that does the maths for AI training quickly.	NVIDIA L4 (22 GB) and RTX PRO 6000 (95 GB) on Google Colab.

Token	A small piece of text (a word or part of a word). The AI reads and writes in tokens.	Sub-word units from Gemma's tokenizer.
Chat template	A fixed layout that marks who is speaking ("user" or "model"), so the AI knows where the question ends and the answer starts.	Gemma 3 uses `<start_of_turn>`; Gemma 4 uses `<|turn>`.
Training / validation set	We split our examples: most are used for studying (training), and some are kept aside as a surprise test (validation).	A 90 % / 10 % split.
Loss	A score for how wrong the AI's answers are. **Lower is better.**	Cross-entropy loss on the answer tokens.
Epoch	One full read-through of all the training examples.	—
Batch size	How many examples the AI looks at before it updates itself.	The number of examples per gradient step.
Learning rate	How big each correction is. Too big and learning is unstable; too small and it is slow.	2×10^{-4}, decreasing linearly over training.
Overfitting	Memorising the study material without understanding it, like a student who memorises past exam papers and fails new questions.	Validation loss rises while training loss keeps falling.
Underfitting	Not having studied enough yet.	Both losses are still high or still clearly falling.

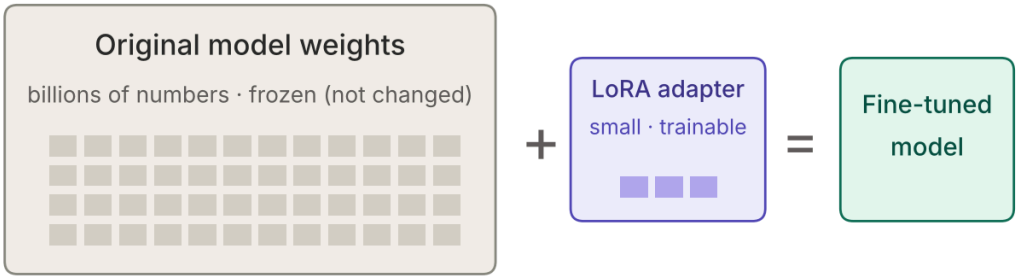


Figure 1. How LoRA works: the big original model stays frozen, and only a small add-on is trained.

TECHNICAL DETAIL: LORA

For a weight matrix W, LoRA learns a low-rank update: $W' = W + (\alpha / r) \cdot B \cdot A$, where A and B are thin matrices with rank r. Adapters were attached to the attention projections (q, k, v, o) and the MLP projections (gate, up, down). A larger r means a bigger, more capable adapter. We set $\alpha = r$, so the update is scaled by 1.

3. How the process works

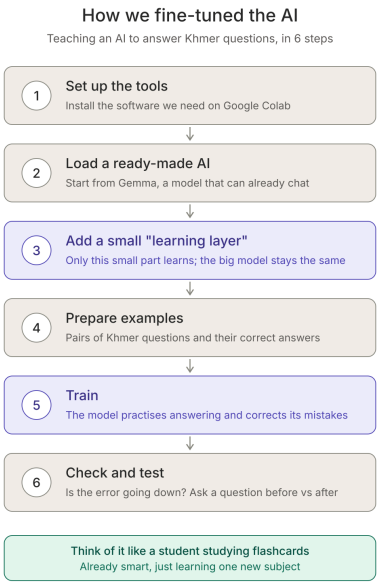


Figure 2. The six-step workflow

1. Set up the tools. We install *Unsloth*, a library that makes fine-tuning about twice as fast and lighter on memory. It uses Hugging Face *Transformers*, *TRL* and *PEFT* underneath.

2. Load a ready-made AI. We download Gemma in compressed 4-bit form and freeze it, so its original knowledge is kept.

3. Add the learning layer. We attach LoRA adapters. In experiment 1 this added 14.9 million trainable numbers (0.35 % of the model).

4. Prepare examples. We clean the data and put every question–answer pair into Gemma's chat layout.

5. Train. The AI reads a question, writes an answer, compares it with the correct answer, and adjusts the adapter slightly. This repeats over and over.

6. Check and test. Every 10 training steps, we measure the error on the surprise-test questions and keep the best version. Finally, we ask the same question with and without the adapter to compare.

An important trick: only grade the answer

Each training example contains both a question and an answer. We **hide the question from the grading**, so the AI is only scored on how well it writes the answer. Otherwise it would also spend effort learning to repeat questions, which isn't useful.

> TECHNICAL DETAIL
>
> `train_on_responses_only()` sets the labels of all user-turn tokens to −100, which the loss function ignores. We checked this in the notebook by decoding training row 100: after masking, only the answer text remains.

4. The data

Experiment 1: ACLEDA Bank FAQ

A custom CSV file of frequently asked questions about ACLEDA Bank's products: debit cards, mobile banking, transfer limits, fees and so on. All text is in Khmer.

Cleaning step	Why	Rows
Load the CSV file	—	521
Rename the columns to `question` and `answer`	The original headers were read incorrectly (`input,output` and `__index_level_0__`)	521
Remove rows with an empty answer	An example without an answer can't teach anything	**514**
Split 90 / 10	Keep a surprise test set	462 train / 52 test

Example pair (translated): **Q:** "What is the ACLEDA JCB Classic debit card?" → **A:** "An electronic card issued by ACLEDA Bank Plc. for paying through e-commerce, POS terminals and ATMs that show the JCB logo, in Cambodia and worldwide, using money from the cardholder's account."

Experiment 2: General Khmer questions and answers

A public dataset by Dr. Rina Buoy (`rinabuoy/text-pairs-khmer-part-2` on Hugging Face) with 179,084 Khmer question–answer pairs on everyday topics: science, health, technology and more. The answers sound like a teacher explaining something simply.

- We randomly picked **5,000 pairs** to keep training time short, and split them into 4,500 for training and 500 for testing.
- *Example (translated):* **Q:** "Where did the 'eight glasses of water a day' rule come from?" → **A:** a short explanation that the recommendation includes water from food, which most people don't realise.

5. The training settings

These settings (called *hyperparameters*) control how training happens. The table shows what we chose and why.

Setting	Exp. 1	Exp. 2	What it means and why we chose it
Model size	4B	12B	General topics need more background knowledge, so we used a bigger model for experiment 2.
Max length	1,024 tokens	2,048 tokens	The longest text an example can have. Experiment 2's explanations are longer.
LoRA rank (r)	8	64	The size of the add-on. With only ~500 examples, a small add-on avoids memorising; 5,000 varied examples need more capacity.
LoRA alpha	8	64	How strongly the add-on influences the model. Kept equal to r, a common safe choice.
Batch size	8	32	Examples per update. The bigger GPU and dataset allowed bigger batches.
Epochs	2	1	A small dataset needs two passes; 4,500 examples are enough for one.
Total steps	116	141	ceil(462 ÷ 8) × 2 = 116 and ceil(4,500 ÷ 32) × 1 = 141. Both runs ended up about the same length.
Learning rate	0.0002, linear decay		A standard value for LoRA. It starts after 5 warm-up steps and shrinks gradually to zero.
Optimizer	AdamW	8-bit AdamW	The method that decides how to adjust the numbers. The 8-bit version saves memory on the large model.
Weight decay	0.01		A small penalty that discourages extreme values and helps prevent overfitting.
Precision	bfloat16		A faster number format that both GPUs support.
Evaluation	Every 10 steps, keep best		We test on the hidden questions regularly and automatically keep the version with the lowest error.

6. Results: did the AI learn?

How to read a loss chart

A loss chart shows the AI's error over training time. There are two lines: **training loss** (error on the material it studies) and **validation loss** (error on the surprise test). The validation line matters most, because it shows how the AI does on *new* questions.

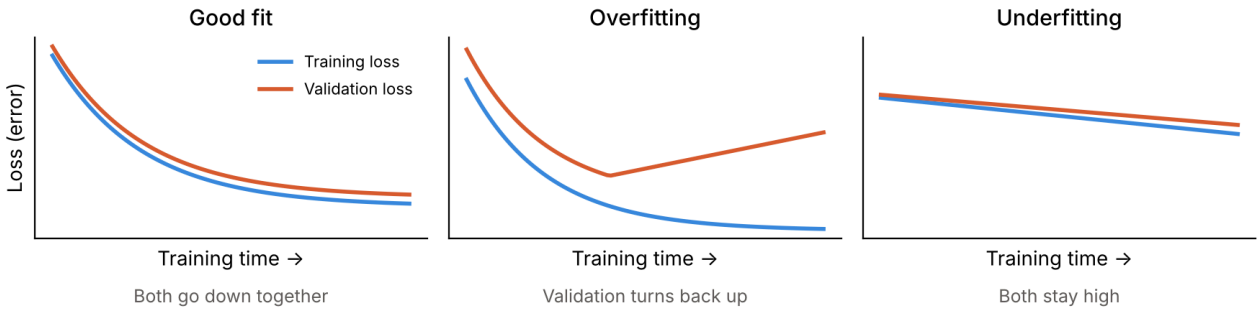


Figure 3. The three typical patterns. We want the first one.

Experiment 1: Bank FAQ

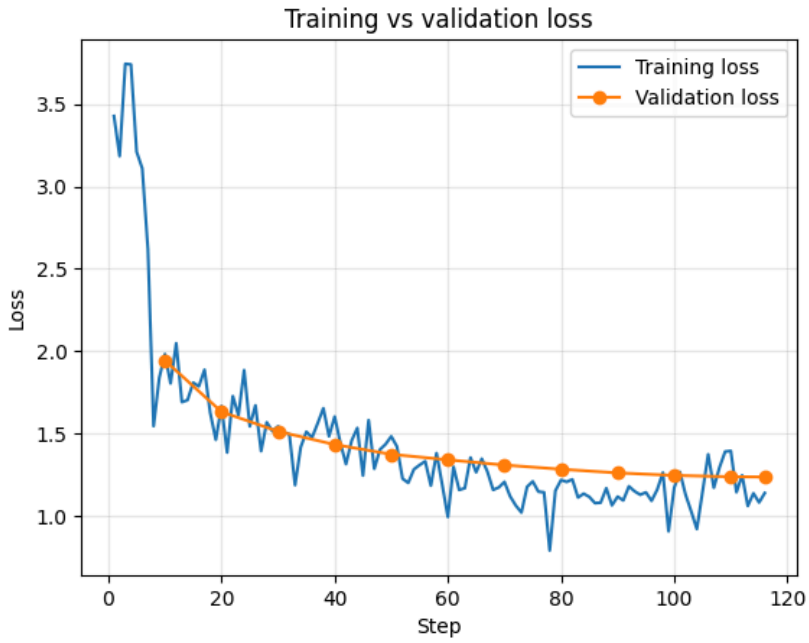


Figure 4. Training and validation loss: Gemma 3 4B on the ACLEDA FAQ

IN PLAIN WORDS

The AI got much better. Its error on unseen bank questions dropped by more than a third and improved at every single check. It never started to get worse, so it was genuinely learning, not memorising. By the end, the improvement had levelled off, meaning it had learned most of what these 462 examples could teach.

TECHNICAL DETAIL

Validation loss fell monotonically from **1.944 → 1.237** (−36 %), with only −0.002 over the last 6 steps (converging). Training loss dropped from ≈ 3.4 to ≈ 1.1–1.2. The steep fall in the first ~10 steps is the model adapting to the format and domain vocabulary. The final train/validation gap is ≈ 0.1, which is small, so there is no overfitting. The jagged training line is normal: each point comes from a single batch of 8 examples, while validation is averaged over 52.

Experiment 2: General Khmer Q&A

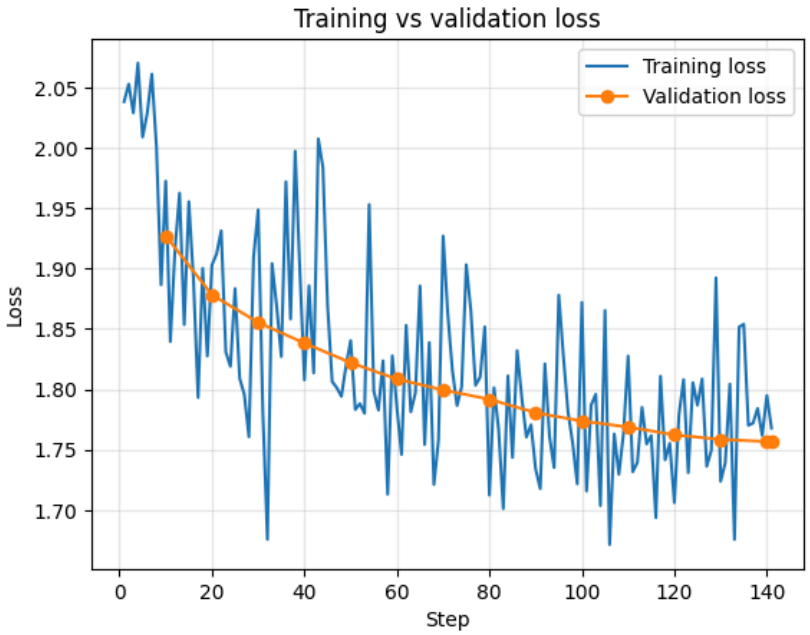


Figure 5. Training and validation loss: Gemma 4 12B on Khmer Q&A pairs. Note that the y-axis only covers 1.70 to 2.05.

IN PLAIN WORDS

The AI also improved steadily on unseen questions, and again never got worse. The training line looks shaky, but that is mostly because the chart is zoomed in a lot. When training stopped, both lines were still going down, so the AI had more to learn. Training longer would probably make it better.

TECHNICAL DETAIL

Validation loss fell monotonically from **1.927 → 1.757** (−9 %) and was still decreasing at step 141. Training loss fluctuates by about ±0.1 because the topics in each batch vary widely. Train and validation losses end almost equal (≈ 1.77 vs 1.76), with no generalisation gap, and both are still trending down. This points to **mild underfitting**: 1 epoch on 2.8 % of the available data is not enough to converge.

Comparing the two

	Validation loss at start	Validation loss at end	Improvement	Diagnosis
Exp. 1: Bank FAQ	1.944	1.237	**−36 %**	**Good fit, nearly finished learning**
Exp. 2: Khmer Q&A	1.927	1.757	−9 %	Mild underfitting, still learning

WHY IS EXPERIMENT 2'S ERROR HIGHER?

It is **not** because that AI is worse. The bank FAQ is narrow and repetitive: the same names and phrases come up again and again, so the answers are very predictable. General questions can be answered correctly in many different ways, so there is always some "error" when the AI's wording differs from the reference answer. **Loss numbers can only be compared fairly within the same dataset.**

7. Before vs after: real answers

Numbers only tell part of the story. Here is what the AI actually said, translated from Khmer, when asked the same question before and after fine-tuning.

Experiment 1: "How many years is the ACLEDA JCB Classic debit card valid?"

Before fine-tuning (original Gemma 3)	After fine-tuning
Wrong and confused. It misspelled the card and bank names, described the card as a *medicine* "valid for 3 years from production", advised consulting a doctor, and switched into **Italian** and **Thai** mid-sentence.	**Correct and clear.** "The ACLEDA JCB Classic debit card is valid for up to 5 years." This matches the bank's FAQ exactly.

After reloading the saved model, we also asked a question about whether ACLEDA Mobile needs an initial deposit and a minimum balance. It answered in fluent, bank-style Khmer, saying no initial deposit is required, only a sufficient balance for transactions.

Experiment 2: "Why can airplanes fly?"

Before fine-tuning (original Gemma 4)	After fine-tuning
A long, textbook-style answer with headings, explaining the "four forces of flight", Bernoulli's principle and Newton's third law. Detailed, with a few misspelled Khmer words.	Short and simple, in the dataset's teacher style: planes fly because of *lift*, created by air flowing under the wings. However, it then **repeated the same sentence over and over** until it hit the length limit.

Experiment 2: "At what temperature does water boil?" (after fine-tuning)

"Water boils at 212 °F (100 °C). That is at sea level. At higher places, the boiling temperature is lower."

Short, correct and helpful, exactly the "teacher explains simply" style the dataset was chosen for.

> **IN PLAIN WORDS**
> In experiment 1 fine-tuning made a dramatic difference: the AI went from inventing nonsense to giving the right bank fact. In experiment 2 the AI already knew the general facts. Fine-tuning mainly changed *how* it answers (shorter and simpler), but it sometimes got stuck repeating itself, which a simple setting can fix (see section 10).

8. Cost and resources

	Experiment 1	Experiment 2
GPU	NVIDIA L4 (22 GB memory)	NVIDIA RTX PRO 6000 Blackwell (95 GB memory)
Training time	8.6 minutes (513 seconds)	8.1 minutes (484 seconds)
Peak GPU memory used	5.2 GB (23 % of the GPU)	16.6 GB (17 % of the GPU)
Trainable numbers	14.9 million of 4.3 billion (0.35 %)	262 million of 12.2 billion (2.15 %)
Size of the saved result	Only the small LoRA add-on is saved (folders `gemma-3/` and `gemma-4/`), not a full copy of the model	

> **IN PLAIN WORDS**
> Thanks to LoRA and compression, a 4-billion-parameter model was customised in under 10 minutes using less than a quarter of a mid-range cloud GPU. This kind of project is affordable for a student or a small team.

9. What this means

For business and decision-makers

- **A Khmer customer-service assistant is realistic.** With a few hundred FAQ entries, the AI learned to answer bank questions correctly in Khmer.
- **Fine-tuning reduces hallucination on known topics.** The original model invented an answer; the trained one gave the real fact.
- **Costs are low**: minutes of GPU time, not weeks. It is easy to retrain when products or policies change.
- **Human checking is still needed.** We tested only a handful of questions. A real deployment would need a larger test set and monitoring.

For technical teams

- LoRA with r = 8 on ~460 examples converged in 2 epochs without overfitting. This is a good starting recipe for small domain datasets.
- For open-domain style transfer, 1 epoch on 4,500 examples underfits. Scale up the data and steps before increasing model size.
- Always set `repetition_penalty` (≈ 1.15) or similar decoding controls when generating with fine-tuned models.
- Response-only masking and keeping the checkpoint with the lowest validation loss worked as intended and should stay in the pipeline.

10. Limitations and issues found

> **NOTEBOOK ISSUE: SWAPPED LABELS**
> In experiment 2's test cells, the cell titled *"Base Model"* actually runs with the LoRA adapter **on** (so it is the fine-tuned model), and the cell titled *"Fine Tuned model"* uses `model.disable_adapter()` (so it is the original model). Section 7 of this report follows what the code actually does. The titles should be swapped in the notebook.

- **Repetition loop:** the fine-tuned Gemma 4 answer looped because no repetition penalty was set in that cell. Experiment 1 used 1.15 and did not have the problem.
- **Small test:** before/after quality was judged on only a few questions. Loss is a useful signal, but it doesn't directly measure whether answers are factually correct.
- **Partial data:** experiment 2 used only 2.8 % of the available dataset.
- **Different models and GPUs:** the two experiments differ in model, data and hardware at the same time, so their results can't be attributed to a single factor.
- **Minor notes:** one notebook comment says "1 %" validation, but the code uses 10 %. The memory report for experiment 2 shows "0 GB for training" because memory usage had already peaked when the model was loaded.

11. Recommended next steps

1. **Fix the notebook labels** in experiment 2 and re-run the comparison with `repetition_penalty = 1.15`.
2. **Train experiment 2 for longer**: use 20,000+ examples or 2–3 epochs, and watch the validation loss for the point where it stops improving.
3. **Build a proper test set** of 30–50 questions with known answers, and score correctness instead of judging single examples.
4. **Compare fairly:** run the same dataset on both models (or the same model with different LoRA ranks) to see which change matters most.
5. **For a real bank assistant:** combine fine-tuning with a document search system (RAG), so answers can be updated without retraining and can cite the FAQ.

Appendix: full numbers

A. Experiment 1 loss values (every 10 steps)

Step	10	20	30	40	50	60	70	80	90	100	110	116
Training	1.983	1.667	1.545	1.604	1.484	0.995	1.207	1.218	1.118	1.172	1.395	1.140
Validation	1.944	1.633	1.514	1.434	1.375	1.341	1.310	1.284	1.262	1.247	1.239	1.237

B. Experiment 2 loss values (every 10 steps)

Step	10	20	30	40	50	60	70	80	90	100	110	120	130	141
Training	1.972	1.903	1.949	1.808	1.840	1.785	1.927	1.712	1.734	1.872	1.828	1.706	1.724	1.768
Validation	1.927	1.878	1.855	1.838	1.822	1.808	1.799	1.792	1.781	1.774	1.769	1.762	1.758	1.757

C. Software and links

- Software: Unsloth 2026.11.3, Transformers 5.17.0, TRL 1.15.0, bitsandbytes 0.50.2, PyTorch 2.14.1, Hugging Face Datasets
- Notebook: colab.research.google.com/drive/13HhuzTUBf2z0j1YvtKlAnOdQCDDcM_6l
- Dataset (exp. 2): huggingface.co/datasets/rinabuoy/text-pairs-khmer-part-2
- Hu et al. (2021), *LoRA: Low-Rank Adaptation of Large Language Models*: arxiv.org/abs/2106.09685
- Unsloth: github.com/unslothai/unsloth